

Operating System

Unit 1 | Assignment 1

Complete Question & Answer Reference

Deadline: 2081-03-26

Subject: Operating System • Level: BCA / BIT

Contents: 6 Questions covering OS definition, system calls, types, structures, open-source OS & shell, and evolution.

Question 1

Define OS. Explain two views of OS in detail.

Answer

Definition of Operating System

An Operating System (OS) is a collection of programs that acts as an interface (intermediary) between a computer user and the computer hardware. Its primary goal is to provide an environment in which a user can execute programs in a convenient and efficient manner while managing the underlying hardware resources. The OS is the one program running at all times on the computer, often referred to as the **kernel**.

Two Views of an Operating System

1. OS as an Extended Machine (User View / Top-Down View)

This perspective focuses on providing a clean, abstract set of resources to application programs and programmers, hiding the complexity of the actual hardware.

- **Hiding Hardware Complexity:** Real hardware (processors, memories, disks) is extremely complicated. For example, the interface specification for a modern SATA hard disk can run over 450 pages. Requiring every programmer to understand such details would make writing code nearly impossible.
- **Providing Abstractions:** The OS hides these messy details and presents a nice, clean, elegant set of abstractions. A prime example is the *file*, which allows users to store data without needing to understand raw disk sectors or track-head movements.
- **Convenience:** The OS turns complex hardware into a simple, beautiful model that is much more user-friendly. Programs obtain services through system calls.

2. OS as a Resource Manager (System View / Bottom-Up View)

From this perspective, the OS acts as a manager for all components of a complex system.

- **Resource Allocation:** A computer system has many resources — CPU time, memory space, file-storage space, and I/O devices. The OS acts as a referee, deciding how to allocate these resources when multiple conflicting requests occur.
- **Efficiency and Fairness:** Facing numerous and often conflicting requests, the OS must ensure that resources are used optimally and fairly, preventing any single user from monopolising the system.
- **Multiplexing (Resource Sharing):** The OS manages resources through two forms:
 - **Time Multiplexing:** Different programs take turns using a resource (e.g., the OS allocates the CPU to one program for a time slice, then switches to another).
 - **Space Multiplexing:** Multiple programs share a resource simultaneously by each getting a portion (e.g., main memory is divided among several running programs).
- **Control Program:** The OS also functions as a control program, managing the execution of user programs to prevent errors and improper use of the computer, especially regarding I/O device control.

Question 2

Explain system call with its handling mechanism.

Answer

What is a System Call?

A system call is the programmatic interface between a running user process and the operating system kernel. It provides the means for a user-level program to request services that are reserved for the OS kernel, such as hardware access or process management. While most programmers interact with the OS through APIs like POSIX (UNIX/Linux) or Win32 (Windows), these APIs typically invoke the actual system calls behind the scenes.

System Call Handling Mechanism (11-Step Process)

The fundamental mechanism involves a transition from **user mode** (mode bit = 1, restricted access) to **kernel mode** (mode bit = 0, full access). Using a *read* operation as an example:

Step 1	Push Parameters	The user program pushes the necessary parameters (e.g., number of bytes to read, buffer address, and file descriptor) onto the stack.
Step 2	Call Library Procedure	The program calls the associated library procedure (e.g., the <code>read()</code> function in the standard C library).
Step 3	Set System-Call Number	The library procedure puts a unique system-call number identifying the specific service into a CPU register where the kernel expects it.

Step 4	Execute TRAP Instruction	The library procedure executes a TRAP instruction (or syscall), which switches the processor from user mode to kernel mode and jumps to a fixed address in the kernel.
Step 5	Examine the Number	The kernel code examines the system-call number stored in the register to identify the requested service.
Step 6	Dispatch to Handler	The kernel dispatches control to the correct system-call handler, typically using a table of pointers indexed by the system-call number.
Step 7	Run Handler	The specific handler routine executes and performs the requested task (e.g., reading from a disk).
Step 8	Return to Library	Once the work is complete, the kernel returns control to the user-space library procedure at the instruction immediately following the TRAP.
Step 9	Return to Program	The library procedure returns to the user program in the same way a normal procedure call returns.
Step 10	Clean Up Stack	The user program increments the stack pointer to remove the parameters that were pushed in Step 1.
Step 11	Continue Execution	The program is now free to continue its execution with the results of the call.

Error Handling

Programs must always check the results of a system call to see if an error occurred. If a call cannot be completed — due to an invalid parameter or hardware error — it typically returns a specific value (such as **-1**) and sets a global variable like **errno** to indicate the nature of the failure.

Question 3

Discuss different types of OS with their examples.

Answer

1. Mainframe Operating Systems

Designed for room-sized computers in major corporate data centers. They manage massive I/O capacity — thousands of disks and millions of gigabytes of data. They offer three service types: Batch (processing routine jobs without user interaction), Transaction Processing (handling high volumes of small requests like bank checks), and Timesharing (multiple remote users running jobs simultaneously).

Examples: OS/390, OS/360

2. Server Operating Systems

Run on large servers (PCs, workstations, or mainframes) and serve multiple users simultaneously over a network. They provide shared resources including print, file, or web hosting services.

Examples: Solaris, FreeBSD, Linux, Windows Server 20xx

3. Multiprocessor Operating Systems

Manage multiple CPUs connected into a single system (also called parallel computers). These are often variations of server OSs with additional features for communication and consistency across CPUs.

Examples: Modern Windows, Linux (multiprocessor builds)

4. Personal Computer (PC) Operating Systems

General-purpose systems designed to provide a good interface for a single user. Used for word processing, spreadsheets, gaming, and internet access. Modern versions all support multiprogramming — dozens of programs active simultaneously.

Examples: Windows 10/11, macOS, Linux, FreeBSD

5. Handheld Computer Operating Systems

Originally called PDAs, today known as smartphones. These devices feature multicore CPUs, various sensors (GPS, cameras, accelerometers), and support vast ecosystems of third-party apps.

Examples: Android, Apple iOS

6. Embedded Operating Systems

Run in devices not generally thought of as computers — microwave ovens, cars, MP3 players, etc. All software is typically stored in ROM. There is no need for protection between applications as no untrusted software runs on them.

Examples: Embedded Linux, QNX, VxWorks

7. Real-Time Operating Systems (RTOS)

Time is a key parameter. Used in industrial process control where actions must occur within strict deadlines. Divided into Hard Real-Time (absolute guarantees, e.g., industrial robots) and Soft Real-Time (occasional deadline misses acceptable, e.g., digital audio or smartphones).

Examples: QNX, RTLinux, HART, eCos

8. Sensor-Node OS

Extremely small, event-driven systems for sensor nodes with very little RAM and limited battery life.

Examples: TinyOS

9. Smart Card OS

The smallest operating systems, running on credit-card-sized devices with severe processing power and memory constraints.

Examples: Java Card OS

10. Batch OS (Historical)

Jobs were collected in batches on magnetic tape and processed sequentially to reduce CPU idle time between jobs.

Examples: FMS (Fortran Monitor System), IBSYS

Question 4a

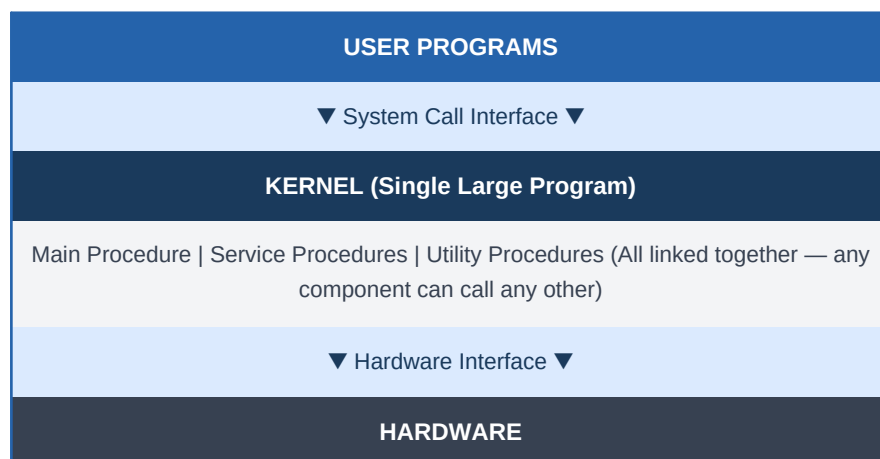
Differentiate between Monolithic and Layered Structure with diagram.

Answer

1. Monolithic Structure

In a monolithic structure, the entire operating system runs as a single, large program in kernel mode. It is written as a collection of procedures all linked together into one executable binary. Every procedure is visible to every other procedure, and any procedure can call any other one.

- **Internal Organization:** Consists of a main program (invokes requested service), service procedures (carry out system calls), and utility procedures (assist service procedures).
- **Advantages:** Highly efficient — minimal overhead for internal communication between components.
- **Disadvantages:** Unwieldy and difficult to understand. A single bug can crash the entire system because everything runs in the same privileged address space.



2. Layered Structure

In a layered structure, the OS is organised into a hierarchy of layers, each built upon the one below it. Each layer uses functions and services of **ONLY** the layers directly below it. Layer 0 is the hardware; the highest layer is the user interface.

- **Advantages:** Simplifies construction and debugging. Errors can be more easily isolated to a specific layer.
- **Disadvantages:** Difficult to define appropriate functionality for each layer. Less efficient — a simple I/O request must pass through multiple layers.

Diagram: THE Operating System Layer Structure

Layer	Function
5 (Highest)	The Operator (User Interface)
4	User Programs
3	Input / Output Management
2	Operator–Process Communication
1	Memory and Drum Management
0 (Lowest)	Processor Allocation and Multiprogramming (Hardware)

Key Differences at a Glance

Feature	Monolithic System	Layered System
Organisation	Single, large executable binary.	Hierarchy of independent levels.
Modularity	Low — every procedure can call any other.	High — layers only call those below them.
Performance	High efficiency due to direct calls.	Lower efficiency due to overhead between layers.
Stability	A bug in one part can crash the whole OS.	More stable; errors are easier to isolate.
Design Logic	Often starts small, grows without a formal plan (e.g., MS-DOS).	Carefully planned top-down approach.

Question 4b

Differentiate between Exokernel and Microkernel.

Answer

1. Microkernel

The basic idea of a microkernel is to achieve high reliability by moving as much functionality as possible OUT of the kernel and into user space.

- **Design Philosophy:** Splits the OS into small, well-defined modules. Only one small module (the microkernel) runs in kernel mode; everything else — device drivers, file systems — runs as ordinary user-space processes.
- **Functionality:** Provides only minimal process and memory management, plus a communication facility (message passing / IPC) to allow client programs to interact with user-space servers.
- **Reliability:** If a component like an audio driver crashes, it will not take down the entire computer because it runs in user space.

- **Examples:** MINIX 3, QNX, L4, Integrity.

2. Exokernel

An exokernel is a program running in kernel mode whose primary job is to partition and allocate resources, rather than providing high-level abstractions.

- **Design Philosophy:** Rather than cloning the actual machine (as in virtual machines) or providing abstractions (as in microkernels), it gives each user a *subset of the actual hardware resources* (e.g., specific disk blocks).
- **Functionality:** Separates management from protection. It allocates resources to virtual machines and then checks all attempts to use them to ensure no machine accesses another's resources.
- **Efficiency:** Saves a layer of mapping. Other designs require the OS to maintain tables to remap disk addresses; the exokernel only tracks which virtual machine has which specific resource — less overhead.
- **Performance:** Allows untrusted software to exercise control over hardware and software resources with less overhead than other designs.

Key Differences at a Glance

Feature	Microkernel	Exokernel
Primary Goal	Reliability through modularity.	Efficiency by partitioning actual resources.
Abstractions	Provides abstractions (files, processes) via user-mode servers.	Avoids abstractions; kernel only handles resource allocation and protection.
Mapping Layer	Adds communication overhead (IPC / message passing).	Saves a layer of mapping by tracking resource assignment directly.
OS Role	Intermediary between client processes and servers.	Referee — ensures machines do not interfere with each other's resources.
Examples	MINIX 3, QNX, L4, Integrity	Exokernel (MIT), Xok/ExOS

Question 5

Explain Open-Source OS and Shell.

Answer

1. Open-Source Operating Systems

Open-source operating systems are those where the source code is made publicly available for anyone to study, modify, and redistribute — rather than being provided only as a compiled binary.

- **Educational Utility:** Serve as excellent learning tools — students can examine the actual code, modify it, and compile their own versions to observe effects.

- **Community Support:** Benefit from a global community of programmers who contribute by debugging, analysing, and suggesting improvements.
- **Security:** Advocates argue they are potentially more secure because 'many eyes' view the code, and bugs are often found and fixed faster.
- **Licensing:** Most are released under the GNU General Public License (GPL), which requires that if binaries are distributed, the source code must also be made available, and any changes must be released under the same license.

OS	Description
Linux	Most famous example, created by Linus Torvalds in 1991. Powers servers, desktops, and embedded systems.
Android	Based on the Linux kernel; open-source platform (AOSP) widely used for mobile devices.
FreeBSD	Derived from BSD UNIX (Berkeley); forms the core of modern macOS systems.
MINIX 3	Highly reliable microkernel-based system focused on fault tolerance and modularity.

2. The Shell

The Shell is a special system program that acts as the **command interpreter** for an operating system. While it is the primary interface between the user and the system, it is important to note that the shell is **NOT part of the OS kernel** — though it makes heavy use of many OS features.

- **How it Works (Fork–Exec Mechanism):**

- The shell displays a prompt (\$ or %) indicating it is ready for input.
- When a user types a command (e.g., date), the shell forks a child process.
- The child process uses the exec() system call to replace its memory with the requested program.
- The shell typically waits for the child to finish before displaying the prompt again.

- **Powerful Shell Features:**

- **Redirection:** Redirect standard output to a file (date > file) or take input from a file (sort < file1).
- **Pipes:** The | operator connects programs — the output of one becomes the input of another (e.g., cat file1 | sort).
- **Background Execution:** Adding & to a command runs it in the background, letting the shell accept the next command immediately.
- **Shell Scripts:** A list of commands saved in a file that the shell executes in order.

- **Common Shell Examples:** sh (Bourne shell), csh (C shell), bash (Bourne Again SHell — default on most Linux systems).
-

Question 6

Discuss the evolution of OS.

Answer

The evolution of operating systems is intimately tied to successive generations of computer hardware.

1st Generation (1945–1955)

Hardware: Vacuum Tubes

- No operating system existed.
- Programming was done in absolute machine code by physically wiring up electrical circuits using plugboards.
- A single group acted as designers, builders, programmers, and operators all at once.
- Machines were used primarily for simple mathematical calculations (e.g., artillery trajectories, sine/cosine tables).

2nd Generation (1955–1965)

Hardware: Transistors & Batch Systems

- The transistor drastically improved reliability — computers (now called mainframes) could be sold to customers.
- Batch Systems introduced: programmers brought card decks to a small computer (IBM 1401) to be read onto magnetic tape, then carried to a large mainframe (IBM 7094) for computing.
- Programs were largely written in FORTRAN or assembly language.
- Typical OS: FMS (Fortran Monitor System) and IBSYS.

3rd Generation (1965–1980)

Hardware: Integrated Circuits (ICs) & Multiprogramming

- ICs gave a massive price and performance advantage.
- IBM introduced the System/360 — a family of compatible machines that could run the same software across different models.
- Multiprogramming: memory partitioned so multiple jobs could reside in memory simultaneously; while one job waited for I/O, the CPU switched to another.
- Spooling: jobs were read from cards directly onto disks as they arrived.
- Timesharing: each user had an online terminal for interactive service.
- UNIX was born: Ken Thompson at Bell Labs developed a stripped-down version of MULTICS, which became UNIX.

4th Generation (1980–Present)

Hardware: LSI/VLSI Circuits & Personal Computers

- LSI (Large Scale Integration) circuits led to the age of the personal computer.

- Early microcomputers used command-line systems like CP/M and MS-DOS.
- The GUI Revolution: Influenced by Xerox PARC research, Apple released the Macintosh with a user-friendly GUI; Microsoft followed with Windows.
- Networking rose: network operating systems (users aware of multiple computers) and distributed operating systems (appear as a single system) emerged.

5th Generation (1990–Present)

Hardware: Mobile Computers & Smartphones

- Focus shifted to handheld devices — from early 'brick' phones to modern smartphones.
- Competition between Symbian and BlackBerry was eventually overtaken by Android (Google, 2008, Linux-based) and Apple's iOS.
- Modern mobile OSs support multicore CPUs, GPS, various sensors, and vast app ecosystems.

— End of Assignment 1 —